



Tema 5: Iluminación, sombreado y texturas

Técnicas de Visualización 3D

Marcos Novalbos Mendiguchía
Marcos.Noalbos@live.u-tad.com

Iluminación

- La luz es uno de los puntos clave en la percepción de las imágenes. Lo que nuestros ojos ven son los rayos de luz reflejados por los objetos.
- Idealmente, un emisor de rayos de luz nos iluminará un objeto. Dependiendo del ángulo de incidencia de esos rayos sobre la superficie, el color percibido variará.
- Los objetos también pueden tener distintas propiedades para su superficie, por lo que una misma luz tendrá distintos resultados sobre el objeto.

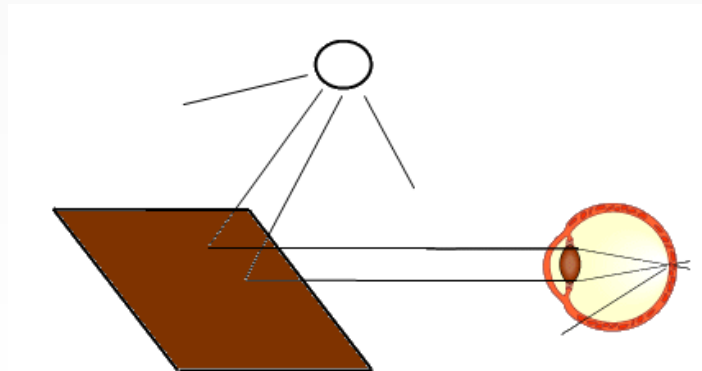
Iluminación



Iluminación

■ Color:

- Inicialmente, la luz visible por el ojo es el resultado de la mezcla de los reflejos de los objetos que haya alrededor, cada uno con sus propiedades.
- El emisor de luz lanzará rayos (fotones) que incidirán sobre un objeto, algunos fotones serán absorbidos, y otros serán reflejados.
- Dependiendo de las propiedades del material, se reflejarán más o menos fotones. Cuanta más cantidad de luz se refleje, parecerá más “brillante”.



Iluminación

■ Color

- La luz también se puede representar como una onda. Dependiendo de sus propiedades, variará el color de la luz.
- En concreto, dependiendo de la longitud de onda, podremos percibir uno u otro color.
- La mezcla de ondas produce que varíen los colores. Presentan propiedades aditivas que permiten que dos emisores de distintos colores puedan mezclar en un momento sus haces de luz para dar un tercer color.

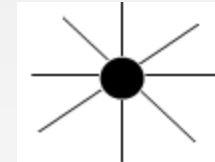


Iluminación

■ Tipos de emisores de luz:

■ Puntos de luz:

- Emiten luz en todas las direcciones.
- No tienen dimensión (tamaño), solo una posición.
- Ej: Sol, bombillas, etc...



■ Fuentes de luz distribuida:

- Emiten luz direccional
- Tiene dimensión
- Ej: Focos, tubos de luz..



Iluminación

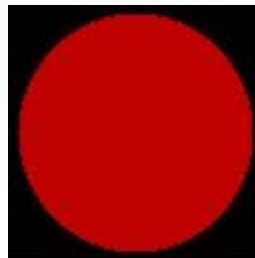
■ Tipos de luz:

- En las aplicaciones informáticas, es necesario descomponer la luz en varios tipos para poder simularla.
- Una simulación real de la física de la luz consume muchos recursos y tiempo, por lo que se tiende a hacer simulaciones visiblemente correctas para aplicaciones de tiempo real.
- Cuatro tipos de luz:
 - Ambiental
 - Difusa
 - Especular
 - Emisiva

Iluminación

■ Luz ambiental

- Sería una media de toda la luz que recibe el objeto desde el emisor de luz. La luz proviene de todos los puntos del espacio (de ahí ambiental).
- Este tipo de luz ilumina por igual todas las facetas del objeto, esta luz se usa como base para iluminar y colorear el objeto. Se entiende como un valor constante.
- La luz ambiental por si sola no añade información a los objetos 3D, por lo que iluminados de esta manera parecen 2D.



Iluminación

■ Luz difusa

- Este es un tipo de luz direccional. Incide sobre el objeto en todos sus puntos con la misma dirección.
- Dependiendo del ángulo de incidencia, creará un efecto de “sombreado”. Esta iluminación si aporta información, por lo que da un aspecto de 3D a los objetos.



Iluminación

■ Luz especular

- La luz especular también es una luz direccional. Añade “brillos” al objeto, y su intensidad depende de una constante del material.
- Esta luz depende del ángulo de visión del observador, puede variar conforme se mueva “el ojo”.



Iluminación

■ Luz Emisora

- Esta luz es distinta de las demás. En este caso el emisor de esta luz es el objeto, y afectará a los objetos que haya alrededor.
- Solo se usa en modelos de iluminación global, e implicaría que a los valores de iluminación de esos objetos haya que sumarles las luces reflejadas.
- Depende de los valores de iluminación del objeto, y de sus propiedades de reflexión.

Modelo de iluminación de Phong

- Este es un modelo simplificado de iluminación, que en la mayoría de los casos da buenos resultados.
- Parte de la definición de luz como la suma de los valores de iluminación ambiental, difusa y especular para cada uno de los puntos de un polígono.
- En este modelo no hay luz emisora desde el objeto, únicamente se refleja la luz que viene de los focos, o reflejada de otros objetos.

$$I_{\text{phong}} = I_{\text{amb}} + I_{\text{diff}} + I_{\text{esp}}$$

Modelo de iluminación de Phong

■ Luz Ambiental

- Es una constante de iluminación -> I_a
- Depende de las propiedades ópticas del objeto -> k_a
(valor empírico dado)

- Por lo tanto :

$$I_{amb} = I_a * k_a$$

Modelo de iluminación de Phong

- Luz Difusa:
 - **N**: normal de la superficie en el punto P
 - **L**: vector de incidencia de la luz
 - **I_d**: intensidad de la fuente de luz
 - **k_d**: coeficiente empírico de reflexión que depende de la longitud de onda de la luz ($0 \leq k_d \leq 1$)
- Por lo tanto el valor de la iluminación en un punto sería:
 - $I_{\text{difusa}} = I_d k_d \cos\theta = I_d k_d (L \cdot N)$

Modelo de iluminación de Phong

- En la expresión anterior, los brillos (iluminación especular) sólo dependen del ángulo Ω entre el observador y el vector de incidencia de la luz.
- En algunos materiales, los brillos sólo se producen cuando la luz incide con un ángulo θ cercano a los 90° con respecto a la normal de la superficie (por ejemplo, el cristal)
- Para modelar esto, podemos sustituir la constante k_e por una función W del ángulo θ
 - $I_{\text{especular}} = I_s W(\theta) \cos^n \Omega = I_s W(\theta) (\mathbf{R} \cdot \mathbf{V})^n$

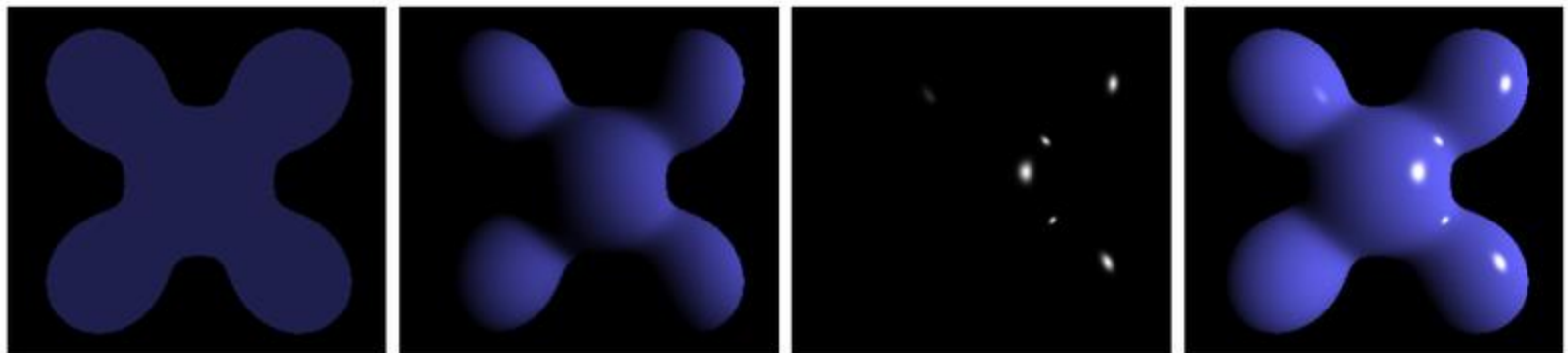
Modelo de iluminación de Phong

- Luz Especular
- Depende de:
 - **R**: vector del rayo de la luz reflejado
 - **V**: vector del visor
 - **I_s**: intensidad de la fuente de luz
 - **k_s**: coeficiente empírico de reflexión especular ($0 \leq k_s \leq 1$)
 - **n**: un índice que simula la rugosidad de la superficie ($1 \leq n < \infty$, 1: mate, ∞ : espejo)
 - Ω : ángulo entre **V** y **R**
- $I_{\text{especular}} = I_s k_s \cos^n \Omega = I_s k_s (\mathbf{R} \cdot \mathbf{V})^n$

Modelo de iluminación de Phong

- Al iluminar con los tres tipos de luz, un objeto con una superficie reflectante quedaría de la siguiente manera:

$$I_{\text{phong}} = I_a * k_a + I_d k_d (L \cdot N) + I_l k_s (R \cdot V)^n$$



Ambient

+

Diffuse

+

Specular

=

Phong Reflection

Modelo de iluminación de Phong

- Cuando la luz viaja por el espacio se va atenuando según una función de la distancia entre el foco de luz y el objeto
 - $f(d) = 1/d^2$
- Una mejor aproximación empírica utiliza la siguiente función de atenuación, donde los coeficientes a_0 , a_1 y a_2 pueden manipularse para conseguir diferentes efectos de iluminación
 - $f(d) = 1/(a_0 + a_1 d + a_2 d^2)$
- La atenuación solo afecta a la luz difusa y a la especular, por lo que el modelo quedaría de la siguiente forma:
 - $I_{\text{phong}} = I_a k_a + f(d)(I_d k_d (L \cdot N) + I_l k_s (R \cdot V)^n)$

Modelo de iluminación de Phong

- **Color:**
 - Con el modelo anterior solo hemos calculado la intensidad de la luz para un punto en escala de grises.
 - Para poder calcular la iluminación usando colores, es necesario especificar los coeficientes de iluminación para cada componente del color, en formato RGB.
 - Se suelen especificar los coeficientes de iluminación como vectores de 3 entradas.

Modelo de iluminación de Phong

- Ejercicio/Ejemplo:
 - Iluminar un punto de un triángulo usando el modelo de iluminación phong
 - Datos:
 - Triángulo: $P1 = (1,1,1)$ $P2 = (0,2,1)$ $P3 = (0,0,1)$
 - Constantes de luz: $k_a = 0.7$ $k_d = 0.9$ $k_s = 0.6$ $n = 10$
 - Punto a iluminar: $(0.33,1,1)$
 - Intensidad de luz ambiente (I_a) = 0.1
 - Punto de luz blanco:
 - Posición = $(1,1,5)$
 - Intensidad especular y difusa (I_s, I_d) = 0.5
 - Posición del visor = $(1,2,5)$

Modelo de iluminación de Phong

■ Paso 1: Iluminación ambiental

- $I_{amb} = I_a * k_a = 0.1 * 0.7 = 0.07$

■ Paso 2: Iluminación difusa

- $I_{difusa} = I_l k_d \cos\theta = I_l k_d (L \cdot N)$

- Calcular N, normal del punto.

- En un plano, todos los puntos tienen la misma normal

Modelo de iluminación de Phong

- Calcular la normal del triángulo:
- 1º, encontrar dos vectores (V1 y V2) pertenecientes al triángulo:
 - $V1 = (P1 - P3) = (1,1,1) - (0,0,1) = (1,1,0)$
 - $V2 = (P2 - P3) = (0,2,1) - (0,0,1) = (0,2,0)$
 - $N = (V1 \times V2) = \begin{vmatrix} i & j & k \\ 1 & 1 & 0 \\ 0 & 2 & 0 \end{vmatrix} = (0,0,2)$
 - Normalizado (0,0,1)

Modelo de iluminación de Phong

- Calcular el vector de incidencia L en el punto. Se calcula como:
 - $L = (P_{luz} - P) = (1, 1, 5) - (0.33, 1, 1) = (0.67, 0, 4)$
- Hay que normalizar:
 - $L_{norm} = (0.67, 0, 4) / 4.055 = (0.165, 0, 0.98)$
- Calcular I_{diff}
 - $I_{diff} = I_d * K_d * (L \cdot N)$
 - $L \cdot N = (0.165, 0, 0.98) * (0, 0, 1) = 0.98$
 - $I_{diff} = 0.5 * 0.7 * 0.98 = 0.44$

Modelo de iluminación de Phong

- Calcular la luz especular
 - $R = 2N(N \cdot L) - L = 2(0,0,1)[0.98] - (0.165,0,0.98)$
 - $R = (-0.165,0,0.98)$
 - $V_{eye} = (0.333,1,1) - (1,2,5) = (-0.15,-0.24,-0.96)$ (normalizado)
 - $R \cdot V_{eye} = -0.86 \Rightarrow$ Negativo, se redondea a 0
 - $I_s k_{spec} (R \cdot V_{eye})^n = 0.5(0.6)(0)^{10} = 0$
- Calcular la iluminación total:
 - $I_{phong} = 0.07 + 0.44 + 0 = 0.51$

Sombreado

- En el cauce gráfico, teníamos dos fases para la iluminación:
 - Iluminación de vértices (procesador de vértices)
 - Sombreado (procesador de fragmentos)
- La iluminación no se realiza sobre cada punto del polígono. Se realiza sobre los vértices, y luego se interpola. A lo largo del polígono.

Sombreado

- Llamamos “etapa de sombreado” al proceso de cálculo de interpolaciones en cada punto del polígono.
- Tres tipos de sombreado:
 - Sombreado plano o de intensidad constante
 - Sombreado de Gouraud
 - Sombreado de Phong

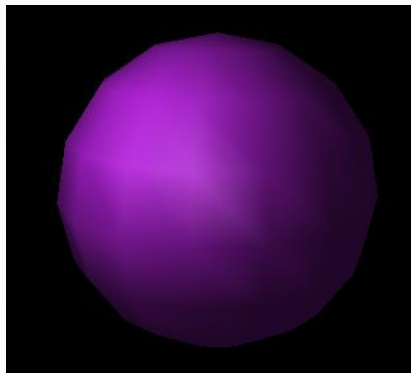
Sombreado plano

- Se calcula el valor de la iluminación sobre un único punto del polígono, y se usa el mismo valor para todos los puntos del mismo.
 - Muy rápido
 - Poco preciso



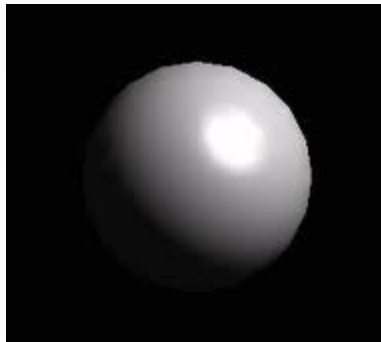
Sombreado Goraud

- Se calcula la iluminación en cada uno de los vértices.
- Para evitar problemas con las aristas, se calcula la normal del vértice como la suma de las normales de los triángulos que lo comparten.
- Calcular la intensidad en los puntos interiores como una interpolación lineal de los valores de los vértices



Sombreado Phong

- Calcular normal en los vértices. Igual que antes, la normal se calcula como la media de las normales de los polígonos adyacentes.
- Calcular la normal de los puntos interiores por interpolación lineal de las de los vértices.
- Calcular la intensidad en cada punto mediante el modelo de iluminación de Phong. Ahora se puede introducir también la reflexión especular



Shaders

- Se necesita guardar la información de normales por vértice:
 - Sombreado plano: Sólo tiene sentido para vértices no compartidos entre caras. No simula "superficie curva".
 - Ej: Cajas, paredes, etc...
 - Sombreado Smooth : Simula una superficie "curva" con pocos polígonos. Los vértices deben estar compartidos entre caras adyacentes
 - Ej: Esferas, objetos cóncavos...

Shaders

- Se debe pasar la información de normal almacenada por cada vértice, y sus posiciones transformadas.

- Vertex Shader:

```
uniform mat4 MVP;
uniform mat4 M;

attribute vec4 pos;
attribute vec4 color;
attribute vec4 normal;

out vec4 fcolor;
out vec4 fpos;
out vec4 fnorm;

void main()
{
    mat4 invT=inverse(transpose(M));
    fnormal = (invT*vnormal);
    fpos=M*pos;

    gl_Position=MVP*pos;
    fcolor=color;
}
```

- Fragment Shader

```
uniform vec3 lightPos;
uniform vec3 eyePos;

in vec4 fpos;
in vec4 fnormal;

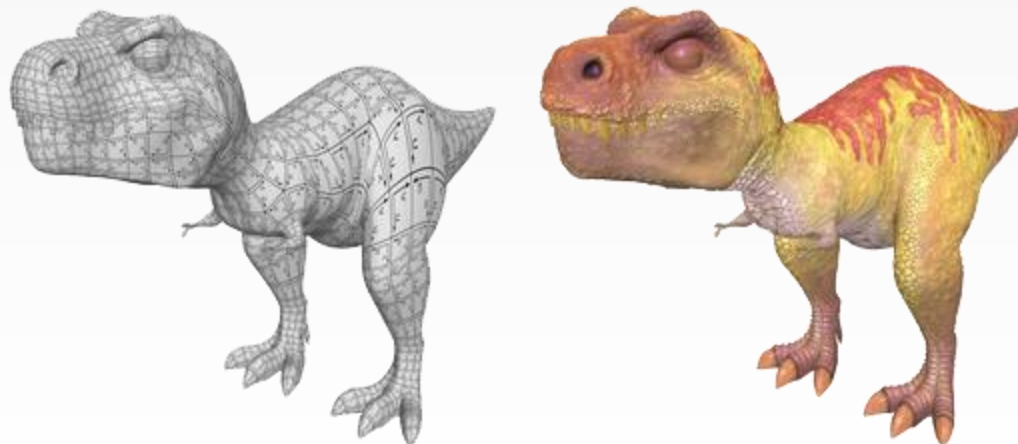
.....
vec3 N=fnormal;
vec3 L=lightPos-fpos;
diffuse=max( dot(L,N),0.0f);

float specular=0.0f;
vec3 R=normalize(reflect(L,N));
vec3 EYE=normalize(fpos.xyz-eyePos);
specular=pow(max(dot(R,EYE),0.0f),shininess);

light=((ambient+diffuse*contribDiff)+specular*contribSpec);
```

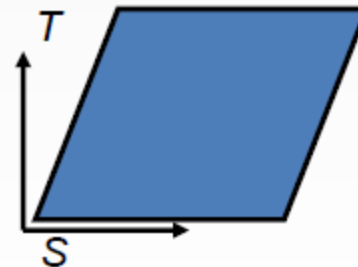
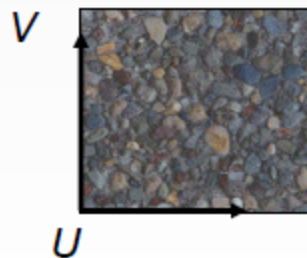
Texturas

- Tenemos una capacidad limitada de generar superficies complejas usando simplemente geometría
- Las imágenes nos dan la ilusión de geometría. Cuando pintamos imágenes en los polígonos hablamos de “texturemapping”.



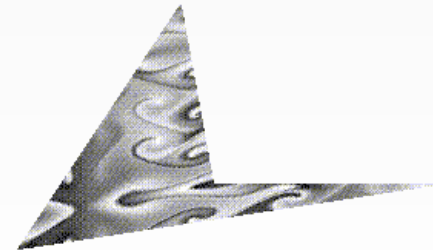
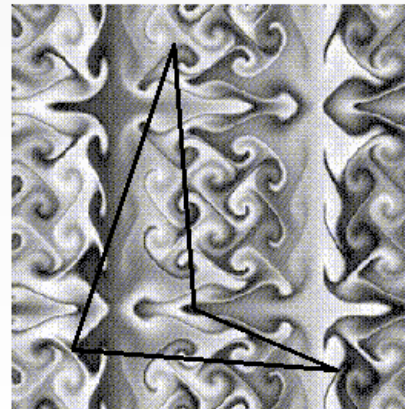
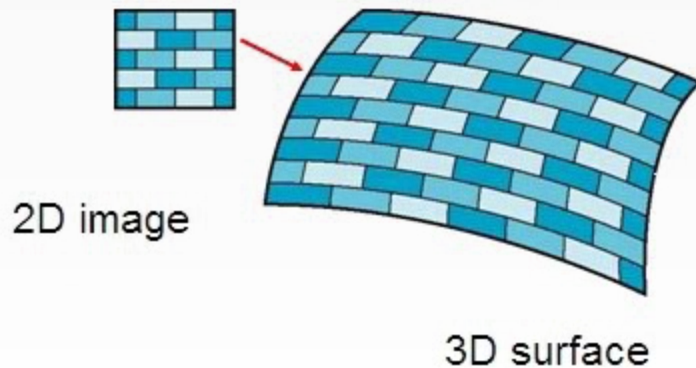
Texture Mapping

- La textura (Texture map) es una imagen en dos dimensiones. Los colores de un pixel en la textura se suele denominar texels.
- Los texels se especifican mediante coordenadas de texturas (u,v)
- Para cada pixel de la pantalla se substituye su color por el texel asociado
- Hacemos corresponder una coordenada (u,v) al espacio de coordenadas del polígono (s, t)

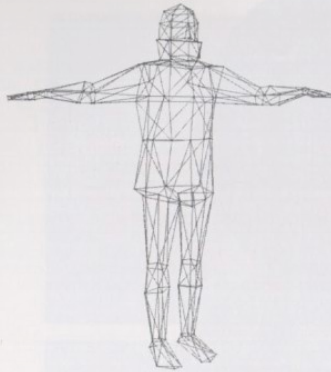


Texture Mapping

- La conversión de (u,v) a (s,t) se realiza dando una coordenada de textura por cada vértice.
- Podemos interpolar la conversión de (u,v) a (s,t) para los puntos entre vértices.

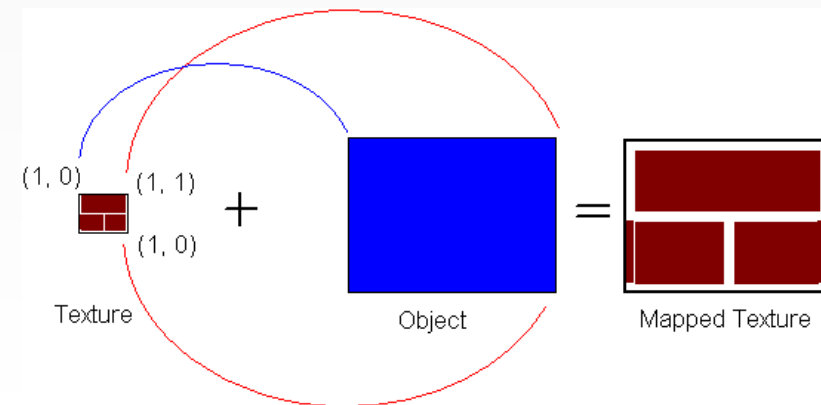
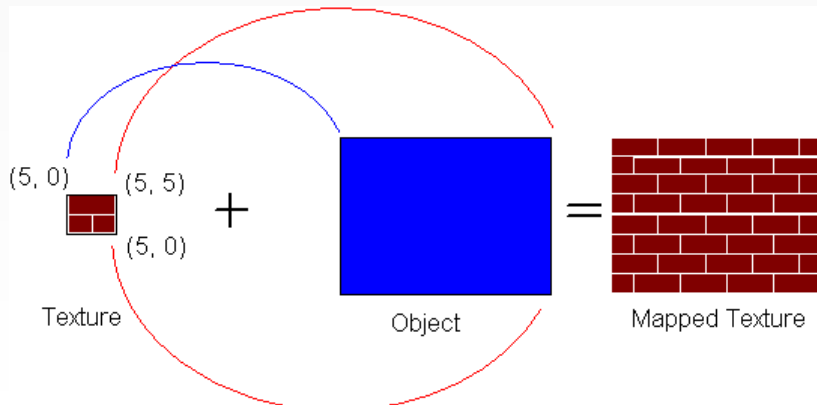


Ejemplo de Texture map en modelos complejos



Coordenadas de texturas

- Cada polígono puede tener coordenadas de objeto y coordenadas de textura
 - Las coordenadas del objeto (sus vértices) dicen donde está éste en el espacio.
 - Las coordenadas de textura describen la posición de un vértice dentro de la textura (texel). Normalmente en el rango 0-1
 - Las coordenadas de texturas se interpolan entre vértices

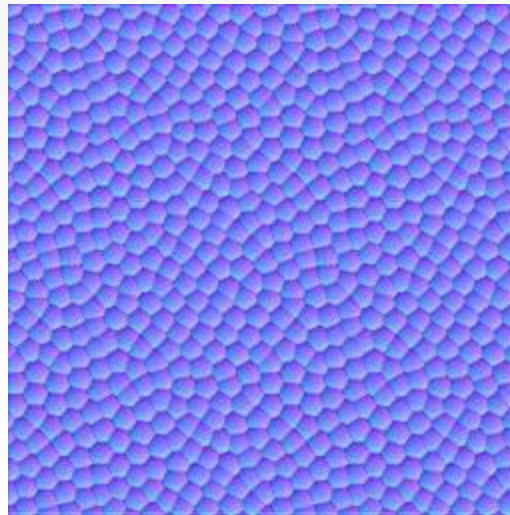


Mapas de profundidad

- Una textura no solo puede almacenar colores, también se puede usar para definir formas.
- Los mapas de profundidad utilizan las texturas para modificar la geometría de las superficies.
 - Utiliza el valor del texel para modificar el valor de la normal en cada pixel
 - Los Texels definen un campo de altura (“heightfield”)
 - “Heightfield” nos permiten modelar superficies rugosas

Bump Mapping

- Bump Mapping es la técnica de mapear “bultos” en polígonos. Usa “Mapas de normales” (Normal Maps), que indican el valor de la normal de cada píxel dentro del polígono .



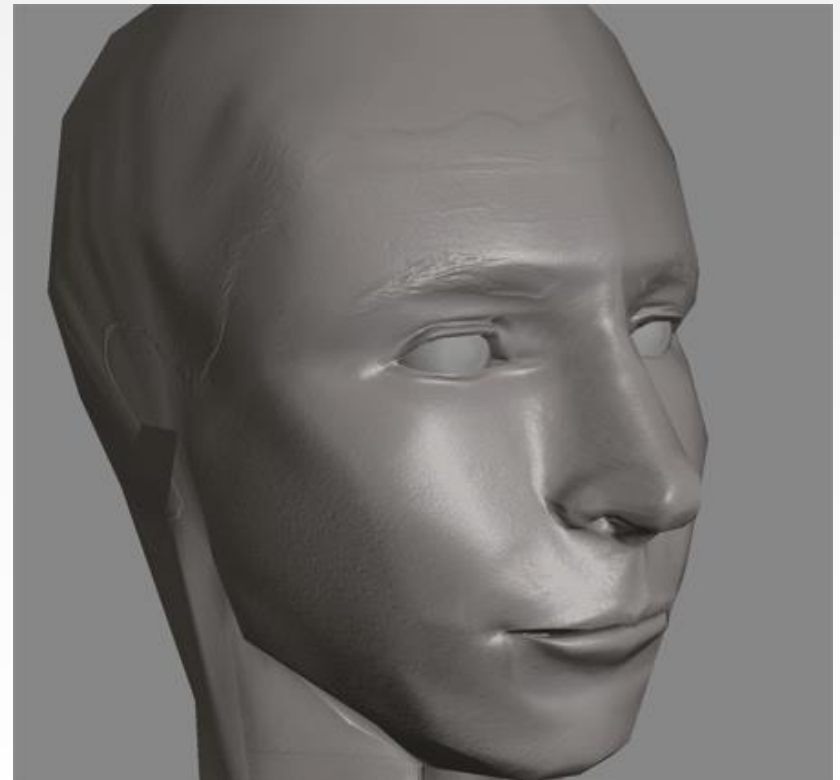
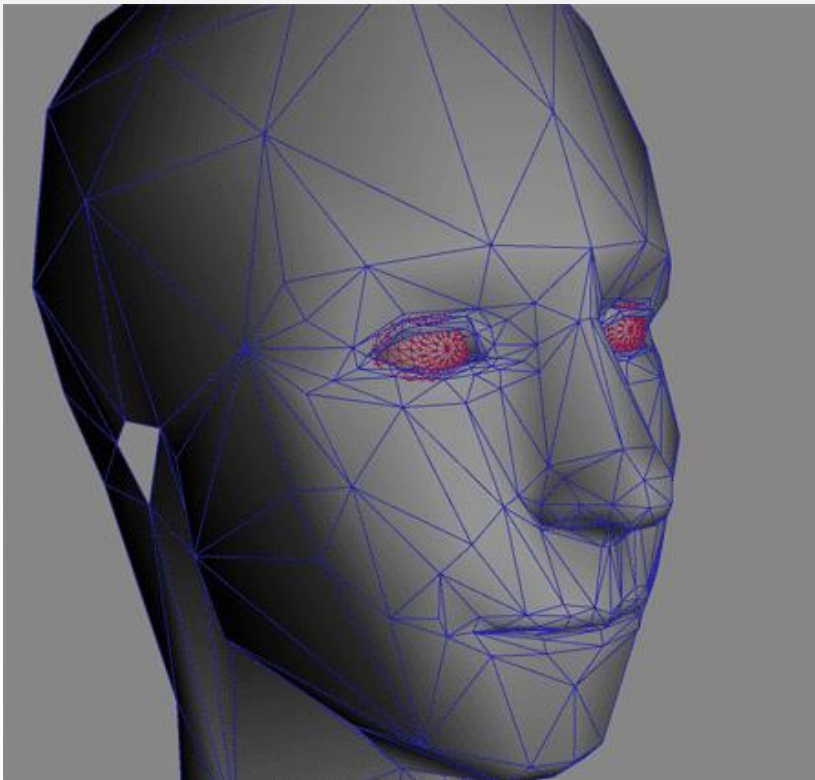
Bump Mapping

- En un polígono todos los puntos deberían tener la misma normal:
 - La textura guarda información sobre las normales dentro del polígono.
 - No guarda información del color o de la imagen



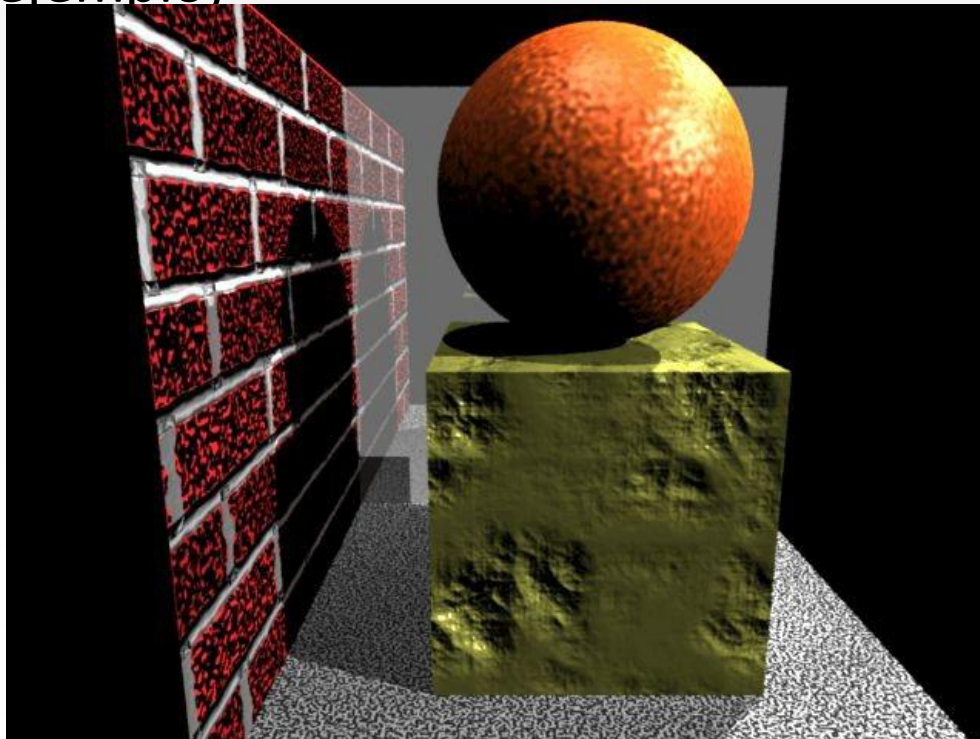
Bump Mapping

- Añade detalle a modelos con pocos polígonos:



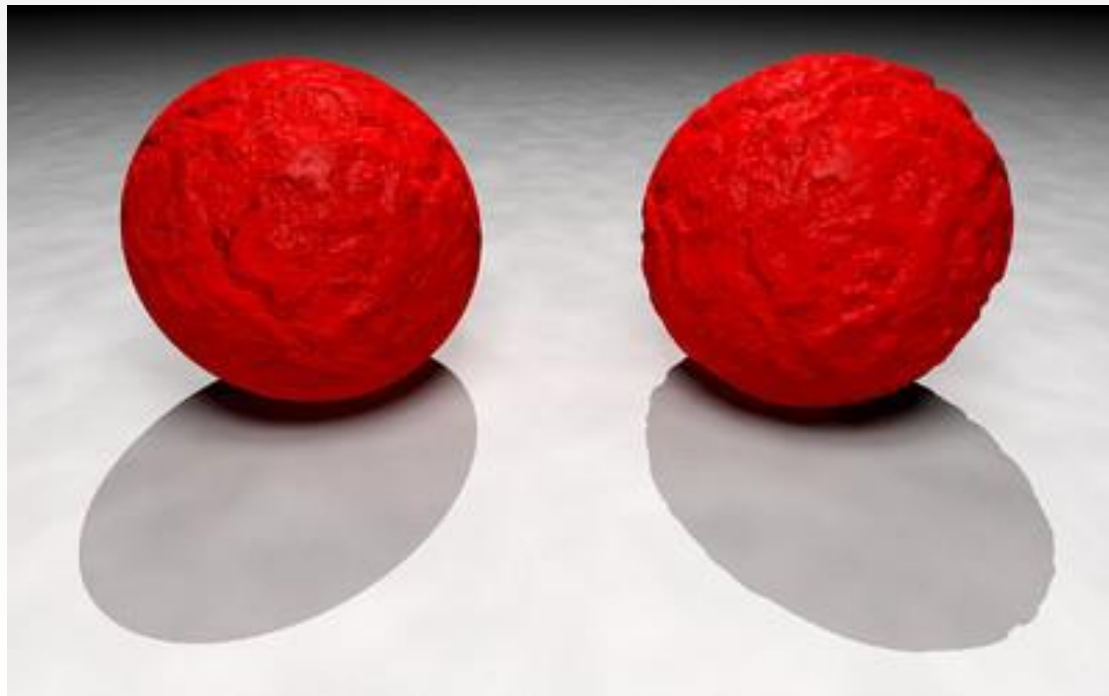
Displacement Mapping

- Los modelos de “Bumpmap” generan normales que son inconsistentes con la geometría real. Podemos tener problemas de sombras (por ejemplo)



Displacement Mapping

- Con la técnica de “Displacement map” alteramos la geometría



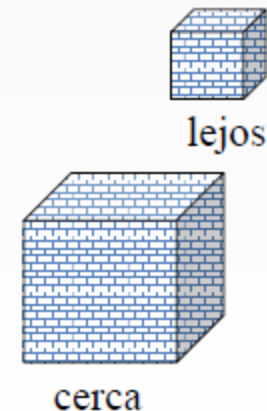
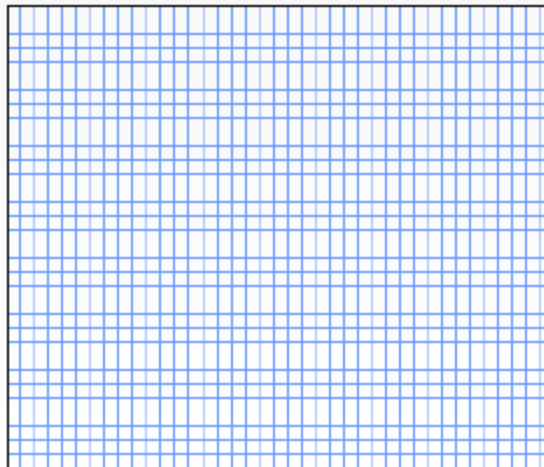
Mipmaps

- Es una técnica “Level Of Detail” (Nivel de detalle) para texturas.
- Define varias texturas a diferentes resoluciones
- Almacena varias texturas, una para cada nivel.



Mipmaps

- Podemos tener varios objetos, unos más cerca que otros del “ojo” del usuario
 - Solo nos interesa que se vea con detalle los más cercanos, los lejanos no lo necesitan.



Cargando una imagen con la librería "stbi"

```
void loadImage(material_t& mat)
{
    int comp = 0;
    stbi_set_flip_vertically_on_load(true);

    mat.imageData = (char*)stbi_load(mat.textureFile.c_str(), &mat.w, &mat.h, &comp, 4);
    if (mat.imageData != nullptr)
    {
        glGenTextures(1, &mat.textId);
        glBindTexture(GL_TEXTURE_2D, mat.textId);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, mat.w, mat.h, 0, GL_RGBA, GL_UNSIGNED_BYTE, mat.imageData);
        mat.enabled = 1;
    }
}
```

Activando el uso de una unidad de textura

```
void materialBind(material_t& mat)
{
    glActiveTexture(GL_TEXTURE0 + mat.textureUnit);
    glBindTexture(GL_TEXTURE_2D, mat.textId);
}
```

Uso de transparencias

- Las texturas de OpenGL soportan un componente llamado alpha , que contiene información sobre la opacidad del texel (píxel de textura) a pintar.
- 0.0 significa totalmente transparente, 1.0 totalmente opaco.
- Con la configuración actual del motor, se ignora el canal alpha , así que independientemente de este valor, se pintan los píxeles totalmente opacos (a este modo de pintado se le suele llamar sólido).
- Es posible definir modos de pintado que tienen en cuenta el valor de este canal alpha.

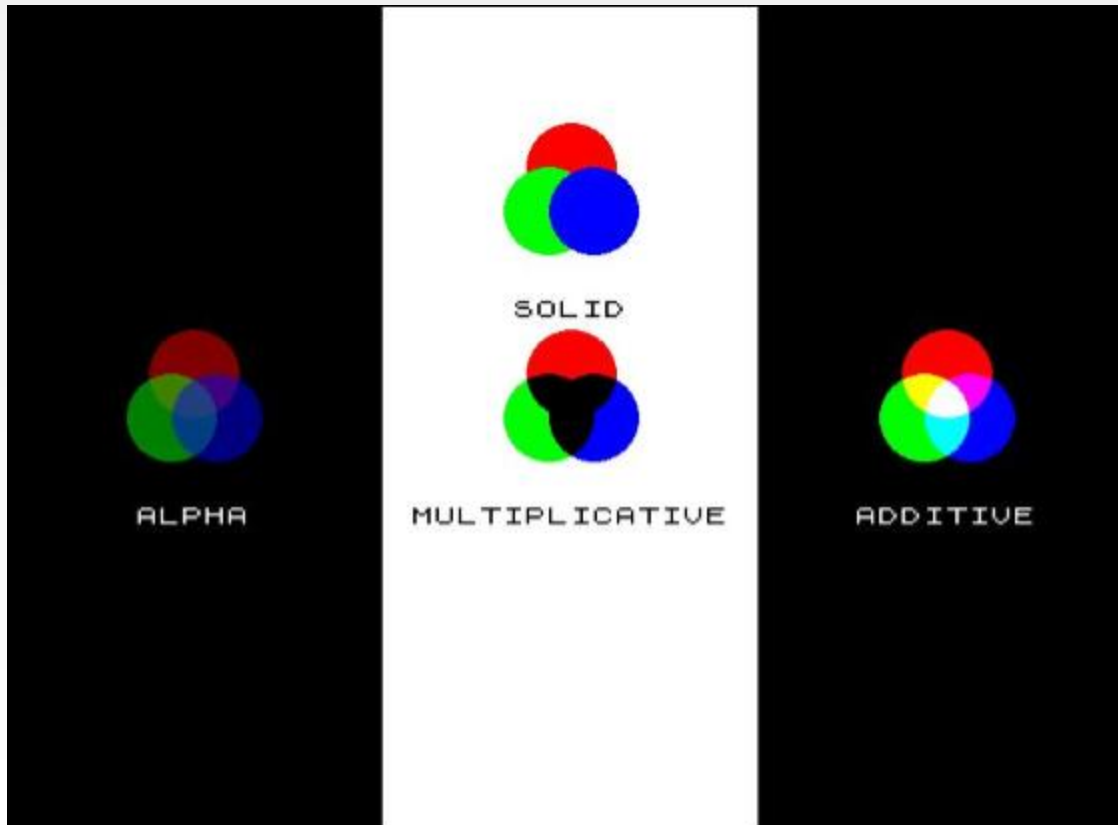
Mezclado de color

- En este apartado trataremos el tema del blending o mezclado de colores.
- El blending es una técnica que consiste en mezclar los colores del píxel que vamos a pintar con los colores del píxel existente en el color buffer en esas coordenadas.
- El píxel a pintar tiene valores RGBA.
- El color buffer únicamente tiene valores RGB.
- Según la forma en que realicemos la mezcla de estos colores, obtendremos distintos efectos de pintado.

Mezclado de colores

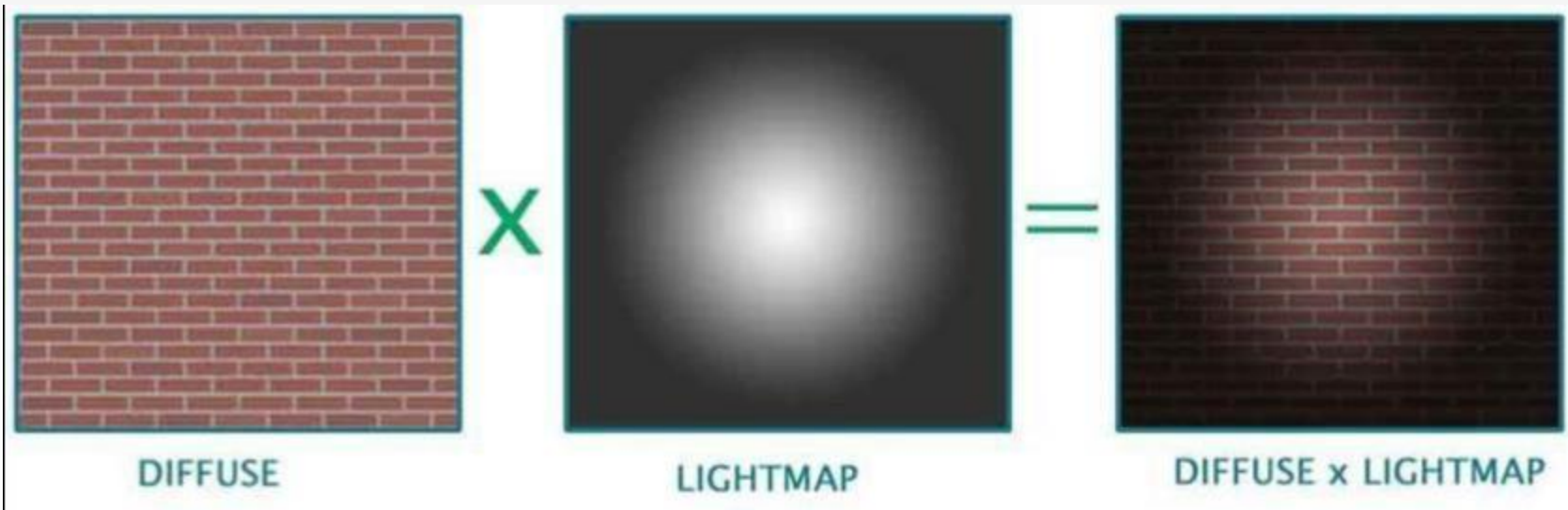
- Los más habituales son estos cuatro:
 - **SOLID:** Los píxeles a pintar reemplazan los píxeles del framebuffer ignorando el valor alpha.
 - **ALPHA:** Los píxeles a pintar se mezclan con los del framebuffer considerando el valor alpha de origen.
 - **MULTIPLICATIVE** o **MODULATIVE:** El color a pintar se multiplica por el color del framebuffer. Produce efectos de oscurecimiento (mezcla "sólida")
 - **ADDITIVE:** El color a pintar se suma al color del framebuffer. Produce efectos de iluminación (mezcla "luz")

Mezclado de colores

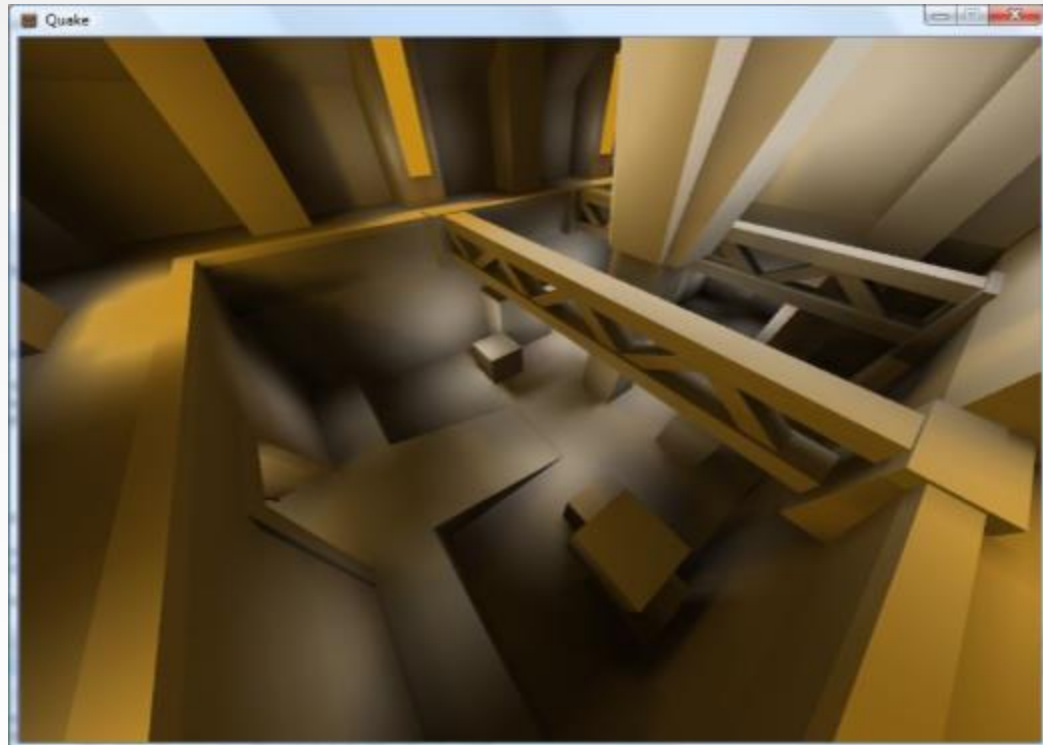


Mezclado de colores

- El modo multiplicativo se utiliza para dibujar lightmaps:



Mezclado de colores



Mezclado de colores



Mezclado de colores

- En OpenGL, el estado GL_BLEND debe estar activado para poder hacer mezclado de colores.
- El modo de pintado se establece con la función glBlendFunc.
 - `void glBlendFunc(GLenum src_factor, GLenum dst_factor)`
- Establece el tipo de mezclado de colores a realizar. La fórmula para calcular el color resultante es:
 - $\text{src_factor} * \text{src_color} + \text{dst_factor} * \text{dst_color}$
- SRC_COLOR sería el color del objeto superior
- DST_COLOR sería el color del objeto inferior

Mezclado de colores

Valores para src_factor y dst_factor:

GL_ZERO

GL_SRC_COLOR

GL_ONE_MINUS_SRC_COLOR

GL_SRC_ALPHA

GL_ONE

GL_DST_COLOR

GL_ONE_MINUS_DST_COLOR

GL_ONE_MINUS_SRC_ALPHA

Orden de pintado

- Si se pintan elementos con transparencia, hay que distinguir entre aquellos objetos en los que consideramos que todos sus píxeles son parcial o totalmente transparentes, y los que tienen píxeles totalmente opacos.
- En ambos casos, se lee la información previamente del depth buffer , para no pintar objetos que se encuentran detrás de sólidos.
- Los elementos transparentes se suelen pintar después de los que tiene elementos sólidos.

Orden de pintado

- Para el primer tipo de transparencia, no escribimos en el depth buffer (de forma que sí se dibujen los elementos que estén detrás).
- Para el otro tipo de transparencia (con algunos píxeles totalmente sólidos), escribimos en el depth buffer si el texel es opaco, y no lo hacemos si el texel es transparente.
- La escritura en el depth buffer se activa con `glDepthMask(GL_TRUE)` (modo por defecto), y se desactiva con `glDepthMask(GL_FALSE)`.
- Para hacerlo a nivel de fragmento, utilizaremos la sentencia `discard` en el fragment shader (¡no escribirá en ningún buffer!).

